📚 Cours complet : Mathématiques de la régression logistique

📑 Sommaire

1. Qu'est-ce que le biais (bias) ?
2. Définitions importantes
    - 1. Sigmoïde (g(z))
    - 2. Fonction de prédiction (h_θ(x))
    - 3. Fonction de coût (J(θ) ou Loss Function)
    - 4. Gradient ($\partial J/\partial \theta_j$)
    - 5. Descente de gradient (Gradient Descent)
    - 6. Learning Rate (α ou alpha)
    - 7. Convergence
    - 8. Produit scalaire ($\theta^T \cdot x$)
    - 9. Logarithme naturel (log ou ln)
    - 10. Entropie croisée (Cross-Entropy)
    - 11. Multicollinéarité
    - 12. Régularisation
    - 13. Surapprentissage (Overfitting)
    - 14. Sous-apprentissage (Underfitting)
3. Notation et variables
4. Objectif et contexte
5. La fonction sigmoïde
6. Le modèle : h_θ(x)
7. La fonction de coût (Loss Function)
8. La dérivée partielle (gradient)
9. Algorithme de descente de gradient
10. Dérivation mathématique du gradient
11. Points importants
12. Résumé des formules clés
13. Exemple complet simplifié
14. Checklist d'implémentation
15. Pour aller plus loin
    - Régression logistique multiclasse (One-vs-All)
    - Régularisation

📖 Qu'est-ce que le biais (bias) ?

Définition simple :

Le **biais (θ_0 ou bias)** est un **terme constant** dans le modèle qui permet de décaler la fonction de décision.

Analogie : Imaginez une balance :

- Les features (x_1, x_2, ...) sont les poids qu'on met sur la balance
- Le **biais** est comme un poids de base qu'on met toujours sur un côté
- Même si toutes les features sont à zéro, le biais peut faire pencher la balance

Pourquoi on en a besoin ?

Exemple visuel :

```
Sans biais :      h = sigmoid(θ₁×x₁ + θ₂×x₂)
                  La fonction passe TOUJOURS par (0, 0.5)

                  Probabilité
                    1 |     /──────
                    0.5|───/
                    0 |/
                      └──────────→ Features
                       0

Avec biais :      h = sigmoid(θ₀×1 + θ₁×x₁ + θ₂×x₂)
                  La fonction peut passer n'importe où !

                  Probabilité
                    1 |     /──────
                    0.5|  /
                    0 |/
                      └──────────→ Features
                       0
```

Exemple concret :

Scénario : Prédire si un étudiant est dans Gryffindor

Sans biais :

```
# Si toutes les notes sont à 0
h = sigmoid(0.3×0 + 0.2×0) = sigmoid(0) = 0.5
→ Toujours 50% de probabilité quand tout est à zéro
```

Avec biais :

```
# Si toutes les notes sont à 0
h = sigmoid(0.5×1 + 0.3×0 + 0.2×0) = sigmoid(0.5) ≈ 0.62
→ 62% de probabilité même avec des notes nulles
→ Le biais "dit" : "Même sans notes, il y a une tendance à être Gryffindor"
```

En résumé :

- **Biais = terme constant** qui ne dépend pas des features
- **Permet de décaler** la fonction de décision
- **Indispensable** pour avoir un modèle flexible
- **Toujours multiplié par 1** (d'où $x_0 = 1$)

📚 Définitions importantes

1. Sigmoïde (g(z))

Définition : La sigmoïde est une fonction qui transforme n'importe quel nombre en une valeur entre 0 et 1.

Formule :

```
g(z) = 1 / (1 + e^(-z))
```

Propriétés :

- Résultat toujours entre 0 et 1
- Si $z \to -\infty \to g(z) \to 0$
- Si $z = 0 \to g(z) = 0.5$
- Si $z \to +\infty \to g(z) \to 1$
- Forme en "S" (courbe sigmoïde)

Pourquoi l'utiliser ?

- Permet d'interpréter le résultat comme une probabilité
- Dérivable partout (important pour le gradient)
- Fonction monotone (croissante)

---

2. Fonction de prédiction (h_θ(x))

Définition : La fonction de prédiction calcule la probabilité qu'un exemple appartienne à la classe 1.

Formule :

```
h_θ(x) = g(θᵀ · x) = sigmoid(θ₀×1 + θ₁×x₁ + θ₂×x₂ + ...)
```

Interprétation :

- `h_θ(x) = 0.9` → 90% de chance d'être dans la classe 1
- `h_θ(x) = 0.1` → 10% de chance (donc 90% d'être dans la classe 0)
- `h_θ(x) = 0.5` → Incertitude maximale

Exemple :

```
theta = [0.5, 0.3, -0.2]
x = [1, 10, 8]

z = 0.5×1 + 0.3×10 + (-0.2)×8 = 1.9
h = sigmoid(1.9) ≈ 0.87

→ 87% de probabilité d'être dans la classe 1
```

---

3. Fonction de coût (J(θ) ou Loss Function)

Définition : La fonction de coût mesure à quel point le modèle fait des erreurs. Plus elle est faible, meilleur est le modèle.

Formule :

```
J(θ) = -1/m × Σ[i=1 à m] [yᵢ × log(h_θ(xᵢ)) + (1-yᵢ) × log(1-h_θ(xᵢ))]
```

Interprétation :

- **Coût faible** (proche de 0) → Modèle fait peu d'erreurs ✅
- **Coût élevé** (grand) → Modèle fait beaucoup d'erreurs ❌
- Objectif : minimiser J(θ)

Pourquoi cette formule ?

- Pénalise fortement les mauvaises prédictions
- Si y=1 et h=0.1 → coût très élevé
- Si y=1 et h=0.9 → coût faible
- C'est la **log-loss** (entropie croisée)

---

4. Gradient (∂J/∂θⱼ)

Définition : Le gradient est la dérivée partielle de la fonction de coût par rapport à un paramètre. Il indique dans quelle direction et de combien ajuster le paramètre pour réduire le coût.

Formule :

```
∂J/∂θⱼ = 1/m × Σ[i=1 à m] (h_θ(xᵢ) - yᵢ) × xᵢⱼ
```

Interprétation :

- **Gradient positif** → Le paramètre est trop grand → Diminuer θ_j
- **Gradient négatif** → Le paramètre est trop petit → Augmenter θ_j
- **Gradient proche de 0** → Paramètre optimal (minimum)

Exemple :

```
# Si ∂J/∂θ₁ = -2.5
# Cela signifie : "Si on augmente θ₁, le coût diminue"
# Donc on fait : θ₁ = θ₁ - α × (-2.5) = θ₁ + α×2.5
```

---

5. Descente de gradient (Gradient Descent)

Définition : Algorithme d'optimisation qui trouve les valeurs optimales des paramètres en les ajustant progressivement dans la direction opposée au gradient.

Principe :

1. Commencer avec des paramètres initiaux (souvent à zéro)
2. Calculer le gradient pour chaque paramètre
3. Ajuster chaque paramètre : `θⱼ = θⱼ - α × gradient`
4. Répéter jusqu'à convergence

Formule de mise à jour :

```
θⱼ = θⱼ - α × ∂J/∂θⱼ
```

Pourquoi "-" ?

- Le gradient pointe vers le maximum
- On veut aller vers le minimum
- Donc on va dans la direction opposée

Analogie : Comme descendre une colline les yeux bandés :

- Le gradient = la pente (direction la plus raide)
- On va dans la direction opposée = on descend
- α = la taille de nos pas

---

6. Learning Rate (α ou alpha)

Définition : Le learning rate contrôle la taille des pas lors de la mise à jour des paramètres.

Rôle :

- **α trop petit** (ex: 0.0001) → Convergence très lente, beaucoup d'itérations
- **α optimal** (ex: 0.01) → Convergence rapide et stable
- **α trop grand** (ex: 1.0) → Risque de divergence, le modèle ne converge pas

Exemple :

```
# Avec α = 0.01
θ₁ = θ₁ - 0.01 × gradient  # Petit pas

# Avec α = 0.1
θ₁ = θ₁ - 0.1 × gradient   # Grand pas (attention à la convergence!)
```

Comment choisir ?

- Commencer par 0.01
- Si trop lent → augmenter légèrement
- Si le coût augmente → diminuer

---

7. Convergence

Définition : La convergence se produit quand le modèle a trouvé les paramètres optimaux. Le coût ne diminue plus significativement.

Signes de convergence :

- Le coût J(θ) ne diminue plus (ou très peu)
- Les gradients sont proches de zéro
- Les paramètres ne changent plus beaucoup

Critères d'arrêt :

- Nombre maximum d'itérations atteint
- Le coût ne diminue plus (seuil fixé, ex: 0.001)
- Les gradients sont très petits (ex: < 0.0001)

---

8. Produit scalaire ($\theta^T \cdot x$)

Définition : Le produit scalaire combine les paramètres avec les features pour obtenir une valeur unique.

Formule :

```
θᵀ · x = θ₀×x₀ + θ₁×x₁ + θ₂×x₂ + ... + θₙ×xₙ
```

Exemple :

```
theta = [0.5, 0.3, -0.2]
x = [1, 10, 8]

produit_scalaire = 0.5×1 + 0.3×10 + (-0.2)×8
                 = 0.5 + 3 - 1.6
                 = 1.9
```

Interprétation :

- Combinaison linéaire des features
- Chaque feature est pondérée par son paramètre
- Résultat : un score qui sera transformé par la sigmoïde

---

9. Logarithme naturel (log ou ln)

Définition : Le logarithme naturel est l'inverse de l'exponentielle. Il transforme les multiplications en additions.

Propriétés importantes :

- `log(1) = 0`
- `log(e) = 1` où e ≈ 2.718
- `log(a × b) = log(a) + log(b)`
- `log(a/b) = log(a) - log(b)`
- Si a < 1 → log(a) < 0 (négatif)
- Si a > 1 → log(a) > 0 (positif)

Pourquoi dans la fonction de coût ?

- Permet de transformer les produits en sommes
- Facilite la dérivation
- Pénalise exponentiellement les mauvaises prédictions

Exemple :

```
log(0.1) ≈ -2.3   # Très négatif (mauvaise prédiction)
log(0.9) ≈ -0.1  # Peu négatif (bonne prédiction)
log(0.5) ≈ -0.7  # Modéré (prédiction incertaine)
```

---

10. Entropie croisée (Cross-Entropy)

Définition : L'entropie croisée est la fonction de coût utilisée en classification. Elle mesure la différence entre la distribution prédite et la distribution réelle.

Pourquoi "entropie" ?

- L'entropie mesure l'incertitude
- Plus la prédiction est certaine (proche de 0 ou 1), plus l'entropie est faible
- Plus la prédiction est incertaine (proche de 0.5), plus l'entropie est élevée

Pourquoi "croisée" ?

- On compare deux distributions : celle prédite (h) et celle réelle (y)
- On mesure leur "croisement" ou divergence

Relation avec la log-loss :

- La log-loss est l'entropie croisée pour la classification binaire
- C'est la même chose, juste un nom différent

---

11. Multicollinéarité

Définition : La multicollinéarité se produit quand deux features ou plus sont très corrélées entre elles.

Problème :

- Les features redondantes n'apportent pas d'information supplémentaire
- Peut rendre le modèle instable
- Difficulté à interpréter les paramètres

Solution :

- Sélectionner les features les moins corrélées
- Supprimer les features avec |corrélation| > 0.7-0.8
- Utiliser la régularisation

---

12. Régularisation

Définition : La régularisation ajoute une pénalité aux grands paramètres pour éviter le surapprentissage (overfitting).

Formule avec régularisation :

```
J(θ) = -1/m × Σ[...] + λ/(2m) × Σ(θⱼ²)
```

Où λ (lambda) contrôle la force de la régularisation.

Effets :

- **λ grand** → Paramètres plus petits → Modèle plus simple
- **λ petit** → Paramètres peuvent être grands → Modèle plus complexe
- **λ = 0** → Pas de régularisation

Pourquoi ?

- Évite que le modèle mémorise les données d'entraînement
- Améliore la généralisation sur de nouvelles données

---

13. Surapprentissage (Overfitting)

Définition : Le surapprentissage se produit quand le modèle mémorise les données d'entraînement au lieu d'apprendre les patterns généraux.

Symptômes :

- Très bon sur les données d'entraînement
- Mauvais sur de nouvelles données
- Paramètres très grands

Solutions :

- Régularisation
- Plus de données d'entraînement
- Moins de features
- Arrêter l'entraînement plus tôt (early stopping)

---

14. Sous-apprentissage (Underfitting)

Définition : Le sous-apprentissage se produit quand le modèle est trop simple et ne capture pas les patterns des données.

Symptômes :

- Mauvais sur les données d'entraînement
- Mauvais sur de nouvelles données
- Modèle trop simple

Solutions :

- Plus de features
- Modèle plus complexe
- Plus d'itérations d'entraînement
- Learning rate plus adapté

🔤 Notation et variables

Avant de commencer, voici les noms complets des variables utilisées :

- **θ (theta)** : Vecteur des paramètres du modèle (poids/bias)
- **θ_0 (theta_0)** : Biais (bias) ou intercept
- **θ_j (theta_j)** : Poids du paramètre j (j = 1, 2, ..., n)
- **x** : Vecteur des features (caractéristiques) d'un exemple
- **x_0 (x_0)** : Toujours égal à 1 (pour le biais)
- **x_i (x_i)** : Feature i (i = 1, 2, ..., n)
- **x_{ij} (x_i_j)** : Valeur de la feature j pour l'exemple i
- **y** : Label réel (valeur cible) - 0 ou 1 pour classification binaire
- **y_i (y_i)** : Label réel pour l'exemple i
- **h_θ(x)** : Fonction de prédiction (hypothèse) - probabilité prédite
- **g(z)** : Fonction sigmoïde
- **z** : Combinaison linéaire = $\theta^T \cdot x$
- **J(θ)** : Fonction de coût (cost function)
- **m** : Nombre d'exemples dans le dataset d'entraînement
- **n** : Nombre de features (caractéristiques)

- **α (alpha)** : Learning rate (taux d'apprentissage)
- **λ (lambda)** : Paramètre de régularisation (optionnel)
- **e** : Nombre d'Euler ≈ 2.71828
- **log** : Logarithme naturel (ln, base e)
- **∂** : Symbole de dérivée partielle

---

1. Objectif et contexte

Problème à résoudre :

On veut faire de la **classification binaire** : prédire si un exemple appartient à la classe 0 ou 1.

Exemple : Prédire si un étudiant appartient à "Gryffindor" (y=1) ou non (y=0) à partir de ses notes.

Différence avec la régression linéaire :

- **Régression linéaire** : Sortie continue (ex: prix = 1000 + 50×taille)
- **Régression logistique** : Sortie = probabilité entre 0 et 1

---

2. La fonction sigmoïde

Définition :

```
g(z) = 1 / (1 + e^(-z))
```

Où `e` est le nombre d'Euler (≈ 2.718).

Propriétés :

- `g(z)` est toujours entre 0 et 1
- Si z → -∞ → g(z) → 0
- Si z = 0 → g(z) = 0.5
- Si z → +∞ → g(z) → 1

Forme de la courbe :

```
g(z)
 1 |      /──────────
   |     /
0.5|───/
   | /
 0 |/
   └──────────→ z
```

Pourquoi la sigmoïde ?

Elle transforme une combinaison linéaire (qui peut être n'importe quel nombre) en une probabilité (toujours entre 0 et 1).

---

3. Le modèle : h_θ(x)

Formule :

```
h_θ(x) = g(θᵀ · x)
```

Où :

- **θ (theta)** = vecteur de paramètres `[θ₀, θ₁, θ₂, ..., θₙ]`
  - **θ_0 (theta_0)** = biais (bias), toujours multiplié par 1
  - **θ_1 à θ_n (theta_1 à theta_n)** = poids pour chaque feature
- **x** = vecteur de features `[x₀=1, x₁, x₂, ..., xₙ]`
  - ⚠ **x_0 (x_0)** = toujours égal à 1 (ajouté artificiellement pour le biais)
    - Les données brutes n'ont pas x_0, on doit l'ajouter : `x = [1] + features_brutes`
    - Exemple : si les données sont `[10, 8]`, on transforme en `[1, 10, 8]`
  - **x_1, x_2, ..., x_n** = valeurs réelles des features
- **$\theta^T \cdot x$** = produit scalaire (dot product) = `θ₀×1 + θ₁×x₁ + θ₂×x₂ + ... + θₙ×xₙ`
- **h_θ(x)** = fonction de prédiction (hypothèse) - la probabilité prédite

Exemple concret :

```
# theta (θ) = paramètres du modèle
theta = [0.5, 0.3, -0.2]  # [θ₀ (bias), θ₁ (poids feature 1), θ₂ (poids
feature 2)]

# Données brutes (2 features)
features_brutes = [10, 8]  # Notes d'Arithmancy et Astronomy

# ⚠ IMPORTANT : Ajouter x₀ = 1 au début pour le biais
x = [1] + features_brutes  # [x₀=1 (ajouté), x₁=10, x₂=8]
# Résultat : x = [1, 10, 8]

# Calculer z = θᵀ · x (produit scalaire)
z = theta[0]×x[0] + theta[1]×x[1] + theta[2]×x[2]
  = θ₀×1 + θ₁×10 + θ₂×8
  = 0.5×1 + 0.3×10 + (-0.2)×8
  = 0.5 + 3 - 1.6
  = 1.9

# Appliquer la sigmoïde g(z)
h_theta_x = g(1.9) = 1/(1+e^(-1.9)) ≈ 0.87

→ Probabilité de 87% que cet étudiant soit dans Gryffindor
```

4. La fonction de coût (Loss Function)

Formule :

```
J(θ) = -1/m × Σ[i=1 à m] [yᵢ × log(h_θ(xᵢ)) + (1-yᵢ) × log(1-h_θ(xᵢ))]
```

Où :

- **J(θ) (J_theta)** = fonction de coût (cost function)
- **m** = nombre d'exemples d'entraînement
- **y_i (y_i)** = label réel (0 ou 1) pour l'exemple i
- **h_θ(x_i) (h_theta_x_i)** = probabilité prédite pour l'exemple i
- **log** = logarithme naturel (ln, base e)
- **Σ** = somme (sigma)

Pourquoi cette formule ?

C'est la **log-loss** (entropie croisée). Elle pénalise fortement les mauvaises prédictions.

Cas 1 : y = 1 (vraie classe = 1)

```
Si h_θ(x) = 0.9 (bonne prédiction) :
  Terme = 1 × log(0.9) ≈ -0.105  → Coût faible ✅

Si h_θ(x) = 0.1 (mauvaise prédiction) :
  Terme = 1 × log(0.1) ≈ -2.303  → Coût élevé ❌
```

Cas 2 : y = 0 (vraie classe = 0)

```
Si h_θ(x) = 0.1 (bonne prédiction) :
  Terme = 0 × log(0.1) + 1 × log(0.9) ≈ -0.105  → Coût faible ✅

Si h_θ(x) = 0.9 (mauvaise prédiction) :
  Terme = 0 × log(0.9) + 1 × log(0.1) ≈ -2.303  → Coût élevé ❌
```

Visualisation :

```
Coût
 5 |     /
   |    /
   |   /
 2 |  /
   | /
 0 |/———————
```

```
    └──────────→ h_θ(x)
    0        1
```

---

5. La dérivée partielle (gradient)

Formule :

```
∂J/∂θⱼ = 1/m × Σ[i=1 à m] (h_θ(xᵢ) - yᵢ) × xᵢⱼ
```

Où :

- **$\partial J/\partial\theta_j$ (dJ_dtheta_j)** = dérivée partielle de J par rapport à θ_j (gradient)
- **j** = indice du paramètre (0, 1, 2, ..., n)
  - j = 0 → biais (θ_0)
  - j = 1, 2, ..., n → poids des features
- **x_{ij} (x_i_j)** = valeur de la feature j pour l'exemple i
- **$h_\theta(x_i)$ (h_theta_x_i)** = probabilité prédite pour l'exemple i
- **y_i (y_i)** = label réel pour l'exemple i
- **m** = nombre d'exemples

Interprétation :

Cette formule nous dit **comment ajuster chaque paramètre θ_j** pour réduire le coût.

Détails :

- `(h_θ(xᵢ) - yᵢ)` = erreur de prédiction
  - Si h > y → erreur positive → on doit diminuer θ_j
  - Si h < y → erreur négative → on doit augmenter θ_j
- `xᵢⱼ` = poids de la feature
  - Si x est grand → ajustement plus important
  - Si x est petit → ajustement plus faible

Exemple :

```
# Pour θ₁ (poids de la feature 1)
# Exemple 1: h=0.8, y=1, x₁=10
Erreur = 0.8 - 1 = -0.2
Contribution = -0.2 × 10 = -2

# Exemple 2: h=0.3, y=0, x₁=5
Erreur = 0.3 - 0 = 0.3
Contribution = 0.3 × 5 = 1.5

# Gradient pour θ₁
∂J/∂θ₁ = (1/2) × (-2 + 1.5) = -0.25
```

```
→ On doit augmenter θ₁ (car gradient négatif)
```

---

6. Algorithme de descente de gradient

Principe :

On met à jour chaque paramètre dans la direction opposée au gradient pour réduire le coût.

Formule de mise à jour :

```
θⱼ = θⱼ - α × ∂J/∂θⱼ
```

Où :

- **θ_j (theta_j)** = paramètre à mettre à jour
- **α (alpha)** = learning rate (taux d'apprentissage), ex: 0.01
- **$\partial J/\partial\theta_j$ (dJ_dtheta_j)** = gradient (dérivée partielle)

Algorithme complet :

```
1. Préparer les données :
   - Pour chaque exemple, ajouter x₀ = 1 au début
   - Exemple : [10, 8] → [1, 10, 8]

2. Initialiser θ = [0, 0, 0, ..., 0]
   - Un paramètre pour le biais (θ₀) + un par feature (θ₁, θ₂, ...)

3. Pour chaque itération :
   a) Calculer h_θ(x) pour tous les exemples
      - z = θ₀×1 + θ₁×x₁ + θ₂×x₂ + ...
      - h = sigmoid(z)
   b) Calculer le gradient ∂J/∂θⱼ pour chaque paramètre
   c) Mettre à jour : θⱼ = θⱼ - α × ∂J/∂θⱼ
   d) (Optionnel) Calculer J(θ) pour suivre la progression

4. Répéter jusqu'à convergence
```

Exemple étape par étape :

Itération 1 :

```
θ = [0, 0, 0]  # Initialisation

# Exemple 1: x=[1, 10, 8], y=1
```

```
z = 0×1 + 0×10 + 0×8 = 0
h = sigmoid(0) = 0.5
Erreur = 0.5 - 1 = -0.5

# Gradient
∂J/∂θ₀ = -0.5 × 1 = -0.5
∂J/∂θ₁ = -0.5 × 10 = -5
∂J/∂θ₂ = -0.5 × 8 = -4

# Mise à jour (α=0.01)
θ₀ = 0 - 0.01 × (-0.5) = 0.005
θ₁ = 0 - 0.01 × (-5) = 0.05
θ₂ = 0 - 0.01 × (-4) = 0.04
```

Itération 2 :

```
θ = [0.005, 0.05, 0.04]

# Exemple 1: x=[1, 10, 8], y=1
z = 0.005×1 + 0.05×10 + 0.04×8 = 0.005 + 0.5 + 0.32 = 0.825
h = sigmoid(0.825) ≈ 0.695
Erreur = 0.695 - 1 = -0.305

# Gradient (plus petit qu'avant, on progresse!)
∂J/∂θ₀ = -0.305 × 1 = -0.305
∂J/∂θ₁ = -0.305 × 10 = -3.05
∂J/∂θ₂ = -0.305 × 8 = -2.44

# Mise à jour
θ₀ = 0.005 - 0.01 × (-0.305) = 0.00805
θ₁ = 0.05 - 0.01 × (-3.05) = 0.0805
θ₂ = 0.04 - 0.01 × (-2.44) = 0.0644
```

À chaque itération, le modèle s'améliore !

---

7. Dérivation mathématique du gradient

Étape 1 : Décomposer la fonction de coût

```
J(θ) = -1/m × Σ [y × log(h) + (1-y) × log(1-h)]
```

Où `h = h_θ(x) = g(z)` et `z = θᵀ · x`.

Étape 2 : Dériver par rapport à θ_j

On utilise la **règle de la chaîne** :

```
∂J/∂θⱼ = ∂J/∂h × ∂h/∂z × ∂z/∂θⱼ
```

Étape 3 : Calculer chaque terme

a) ∂J/∂h :

```
∂J/∂h = -1/m × [y/h - (1-y)/(1-h)]
     = -1/m × [(y - h) / (h(1-h))]
```

b) ∂h/∂z :

```
h = g(z) = 1/(1+e^(-z))

∂h/∂z = e^(-z) / (1+e^(-z))²
      = h × (1-h)
```

c) ∂z/∂θⱼ :

```
z = θ₀×1 + θ₁×x₁ + ... + θⱼ×xⱼ + ...

∂z/∂θⱼ = xⱼ
```

Étape 4 : Combiner

```
∂J/∂θⱼ = -1/m × [(y-h)/(h(1-h))] × [h(1-h)] × xⱼ
       = -1/m × (y-h) × xⱼ
       = 1/m × (h-y) × xⱼ
```

Résultat final :

```
∂J/∂θⱼ = 1/m × Σ (h_θ(xᵢ) - yᵢ) × xᵢⱼ
```

---

8. Points importants

1. Normalisation des features

Avant l'entraînement, normaliser les features pour éviter que certaines dominent :

```
x_normalized = (x - min) / (max - min)  # Entre 0 et 1
```

2. Choix du learning rate

- **Trop petit** (ex: 0.0001) → convergence lente
- **Trop grand** (ex: 1.0) → peut diverger
- **Optimal** : généralement entre 0.001 et 0.1

3. Nombre d'itérations

- **Trop peu** → modèle non entraîné
- **Trop** → risque de surapprentissage
- **Solution** : arrêter quand le coût ne diminue plus

4. Initialisation de θ

- Initialiser à zéro est souvent suffisant
- Initialisation aléatoire peut aider

---

9. Résumé des formules clés

```
1. Sigmoïde :          g(z) = 1/(1+e^(-z))
   où z = θᵀ · x (produit scalaire)

2. Prédiction :        h_θ(x) = g(θᵀ · x)
   où h_θ(x) = probabilité prédite entre 0 et 1

3. Fonction de coût :  J(θ) = -1/m × Σ[y×log(h) + (1-y)×log(1-h)]
   où J(θ) = coût, m = nombre d'exemples, y = label réel, h = probabilité
prédite

4. Gradient :          ∂J/∂θⱼ = 1/m × Σ(h - y) × xⱼ
   où ∂J/∂θⱼ = dérivée partielle, h = probabilité prédite, y = label réel,
xⱼ = feature j

5. Mise à jour :       θⱼ = θⱼ - α × ∂J/∂θⱼ
   où θⱼ = paramètre, α = learning rate, ∂J/∂θⱼ = gradient
```

Variables dans les formules :

- **θ (theta)** = paramètres du modèle
- **θ_0 (theta_0)** = biais
- **θ_j (theta_j)** = poids du paramètre j
- **x** = vecteur de features
- **x_0 (x_0)** = toujours 1 (pour le biais)
- **x_j (x_j)** = feature j

- **y** = label réel (0 ou 1)
- **h** ou **h_θ(x)** = probabilité prédite
- **z** = combinaison linéaire = $\theta^T \cdot x$
- **g(z)** = fonction sigmoïde
- **J(θ)** = fonction de coût
- **m** = nombre d'exemples
- **α (alpha)** = learning rate
- **e** = nombre d'Euler ≈ 2.718
- **log** = logarithme naturel

---

10. Exemple complet simplifié

Données brutes :

```
X_raw = [[10, 8], [12, 9], [5, 3]]  # 3 exemples, 2 features (sans le
biais)
y = [1, 1, 0]                        # Labels
```

⚠ IMPORTANT : Ajouter x_0 = 1 pour le biais

Avant de commencer, on doit **ajouter artificiellement** $x_0 = 1$ au début de chaque exemple pour le biais (θ_0) :

```
# Données transformées (avec x₀=1 ajouté)
X = [
    [1, 10, 8],   # Exemple 1 : [x₀=1 (ajouté), x₁=10, x₂=8]
    [1, 12, 9],   # Exemple 2 : [x₀=1 (ajouté), x₁=12, x₂=9]
    [1, 5, 3]     # Exemple 3 : [x₀=1 (ajouté), x₁=5, x₂=3]
]
```

Pourquoi x_0 = 1 ?

- x_0 est toujours égal à 1, il sert uniquement à multiplier θ_0 (le biais)
- Cela permet d'avoir un terme constant dans notre modèle : `θ₀×1 + θ₁×x₁ + θ₂×x₂`
- Sans ce 1, on ne pourrait pas avoir de biais indépendant des features

Étape 1 : Initialiser

```
# theta (θ) = paramètres du modèle
# On a 3 paramètres : θ₀ (bias) + θ₁ (feature 1) + θ₂ (feature 2)
theta = [0, 0, 0]  # [θ₀ (bias), θ₁ (poids feature 1), θ₂ (poids feature
2)]

# alpha (α) = learning rate
alpha = 0.01
```

Étape 2 : Calculer les prédictions

```
# Exemple 1: x = [x₀=1, x₁=10, x₂=8]
# (x₀=1 a été ajouté, les vraies features sont x₁=10 et x₂=8)
z_1 = theta[0]×x[0] + theta[1]×x[1] + theta[2]×x[2]
    = theta[0]×1 + theta[1]×10 + theta[2]×8
    = 0×1 + 0×10 + 0×8
    = 0
h_1 = sigmoid(z_1) = sigmoid(0) = 0.5  # h_θ(x₁) = probabilité prédite

# Exemple 2: x = [x₀=1, x₁=12, x₂=9]
z_2 = theta[0]×1 + theta[1]×12 + theta[2]×9 = 0
h_2 = sigmoid(z_2) = sigmoid(0) = 0.5  # h_θ(x₂)

# Exemple 3: x = [x₀=1, x₁=5, x₂=3]
z_3 = theta[0]×1 + theta[1]×5 + theta[2]×3 = 0
h_3 = sigmoid(z_3) = sigmoid(0) = 0.5  # h_θ(x₃)
```

Étape 3 : Calculer le gradient

```
# m = 3 (nombre d'exemples)

# Pour θ₀ (theta_0, le biais) - x₀=1 toujours pour tous les exemples
dJ_dtheta_0 = (1/m) × [(h_1 - y_1)×x_0_1 + (h_2 - y_2)×x_0_2 + (h_3 -
y_3)×x_0_3]
            = (1/3) × [(0.5-1)×1 + (0.5-1)×1 + (0.5-0)×1]
            = (1/3) × [-0.5 - 0.5 + 0.5]
            = -0.167

# Pour θ₁ (theta_1, poids de la feature 1)
dJ_dtheta_1 = (1/m) × [(h_1 - y_1)×x_1_1 + (h_2 - y_2)×x_1_2 + (h_3 -
y_3)×x_1_3]
            = (1/3) × [(0.5-1)×10 + (0.5-1)×12 + (0.5-0)×5]
            = (1/3) × [-5 - 6 + 2.5]
            = -2.833

# Pour θ₂ (theta_2, poids de la feature 2)
dJ_dtheta_2 = (1/m) × [(h_1 - y_1)×x_2_1 + (h_2 - y_2)×x_2_2 + (h_3 -
y_3)×x_2_3]
            = (1/3) × [(0.5-1)×8 + (0.5-1)×9 + (0.5-0)×3]
            = (1/3) × [-4 - 4.5 + 1.5]
            = -2.333
```

Étape 4 : Mettre à jour les paramètres

```
# alpha (α) = 0.01 (learning rate)

theta[0] = theta[0] - alpha × dJ_dtheta_0
```

```
           = 0 - 0.01 × (-0.167)
           = 0.00167

theta[1] = theta[1] - alpha × dJ_dtheta_1
           = 0 - 0.01 × (-2.833)
           = 0.02833

theta[2] = theta[2] - alpha × dJ_dtheta_2
           = 0 - 0.01 × (-2.333)
           = 0.02333
```

Répéter pour plusieurs itérations jusqu'à convergence.

---

11. Checklist d'implémentation

- ☐ Implémenter la sigmoïde : `g(z) = 1/(1+e^(-z))`
- ☐ Implémenter la prédiction : $h_\theta(x) = g(\theta^T \cdot x)$
- ☐ Implémenter la fonction de coût : $J(\theta) = -1/m \times \Sigma[...]$
- ☐ Implémenter le gradient : $\partial J/\partial\theta_j = 1/m \times \Sigma(h-y)\times x_j$
- ☐ Implémenter la mise à jour : $\theta_j = \theta_j - \alpha \times gradient$
- ☐ Boucle d'entraînement avec plusieurs itérations
- ☐ Normaliser les features avant l'entraînement
- ☐ Tracer le coût pour vérifier la convergence

---

12. Pour aller plus loin

Régression logistique multiclasse (One-vs-All)

Pour classifier plusieurs classes (ex: 4 maisons de Poudlard), on entraîne un modèle par classe :

- Modèle 1 : Gryffindor vs (tous les autres)
- Modèle 2 : Hufflepuff vs (tous les autres)
- Modèle 3 : Ravenclaw vs (tous les autres)
- Modèle 4 : Slytherin vs (tous les autres)

Pour prédire, on choisit la classe avec la probabilité la plus élevée.

Régularisation

Pour éviter le surapprentissage, on peut ajouter un terme de régularisation :

```
J(θ) = -1/m × Σ[...] + λ/(2m) × Σ(θⱼ²)
```

Où :

- **λ (lambda)** = paramètre de régularisation (contrôle la force de la régularisation)
- Plus λ est grand → plus on pénalise les grands paramètres → modèle plus simple

- Plus λ est petit → moins de régularisation → modèle plus complexe

---

Bon courage pour l'implémentation ! 🚀